TRABAJO PRÁCTICO: ENTREGA CONTINUA Y RESILIENCIA DEL SOFTWARE

Unidad 4 – Semana 3

Metodología de Sistemas II – Tecnicatura Universitaria en Programación a Distancia – UTN

Apellido y nombre:	Figueroa, Ignacio – DNI 45.406.120
Comisión:	C4 - A26

Parte 1: Integración Continua (CI)

1. Filosofía Fail-Fast

"Fail-Fast" significa que el pipeline tiene que **fallar lo antes posible**. Las validaciones rápidas y baratas (lint, tests unitarios) van primero, y apenas una falla el pipeline se corta y avisa. No tiene sentido compilar, empaquetar y correr tests de integración de 20 minutos si el código ni siquiera pasa el linter.

- **Agilidad del equipo:** el desarrollador se entera del error en uno o dos minutos, mientras todavía tiene el cambio fresco en la cabeza, y lo corrige enseguida.
- **Ahorro de recursos:** no se gasta tiempo de servidores en etapas que no van a servir, porque el código ya tenía un error. En servicios pagos, como los minutos de GitHub Actions, también es plata.
- **Rama principal estable:** el error se frena antes de integrarse y no afecta al resto del equipo.

2. Anatomía del pipeline

Etapa	Qué valida	Qué riesgo mitiga
Lint & Static Analysis	Estilo y formato del código, errores comunes (variables sin usar, imports de más), complejidad ciclomática y código duplicado, sin ejecutar el programa.	Código difícil de mantener, bugs simples que se ven sin ejecutar y deuda técnica.
Unit Tests	Que cada función o clase haga lo que tiene que hacer, de forma aislada, y que la cobertura no baje del mínimo acordado.	Errores de lógica y regresiones, es decir, romper algo que antes funcionaba.
Security Scan	Dependencias con vulnerabilidades conocidas (por ejemplo con pip-audit o npm audit), contraseñas o claves escritas en el código y patrones inseguros como inyección SQL.	Que se publique software con agujeros de seguridad o que se filtren credenciales.

3. Pipeline as Code

Significa que todo el pipeline (qué etapas tiene, en qué orden, con qué herramientas y cuándo se ejecuta) está definido en un archivo de texto, normalmente YAML. Es el "cerebro" porque el servidor de CI no tiene nada configurado a mano: lee ese archivo y hace exactamente lo que dice. Un ejemplo simple con GitHub Actions:

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  calidad:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
      - run: pip install -r requirements-dev.txt
      - run: flake8 src tests          # 1. Lint (si falla, corta acá)
      - run: pytest --cov              # 2. Tests unitarios
      - run: pip-audit -r requirements.txt   # 3. Security scan
```

Que esté dentro del mismo repositorio es buena práctica porque:

- **Queda versionado:** se ve quién cambió el pipeline, cuándo y por qué, y se puede volver atrás.
- **Va de la mano con el código:** cada versión del código tiene la versión del pipeline que le corresponde.
- **Se revisa como cualquier cambio:** las modificaciones al pipeline pasan por un Pull Request.
- **Es reproducible:** si se cambia de servidor, el pipeline se recrea igual a partir del archivo.

Parte 2: Versionado Semántico (SemVer)

4. El lenguaje del cambio

SemVer usa el formato **MAYOR.MENOR.PARCHE**. Partiendo de la v2.4.1:

Escenario	Nueva versión	Por qué
A	v2.4.2	Es una corrección de un error que no cambia cómo se usa el sistema, así que sube el PARCHE.
B	v2.5.0	Es una funcionalidad nueva compatible con lo anterior, así que sube el MENOR y el parche vuelve a 0.
C	v3.0.0	Elimina una función y cambia la autenticación, y los sistemas que dependen de este se rompen si no se adaptan. Es un cambio incompatible, así que sube el MAYOR y los otros dos vuelven a 0.

5. Tags y disparadores

Un tag en Git es una etiqueta fija que apunta a un commit específico, por ejemplo v2.5.0. A diferencia de una rama, no se mueve. El pipeline se puede configurar para que se dispare **solo cuando se sube un tag** que cumpla un patrón:

```
on:
  push:
    tags:
      - "v*.*.*"      # solo tags con formato SemVer

# Flujo:
git tag -a v2.5.0 -m "Agrega exportar a PDF"
git push origin v2.5.0      # esto dispara el deploy a producción
```

Al recibir el tag, el pipeline construye el artefacto de ese commit exacto, lo etiqueta con el número de versión (por ejemplo, la imagen app:2.5.0) y lo despliega en producción. Así, publicar una versión es una decisión explícita (crear el tag) y queda registrado qué código exacto está en producción.

Parte 3: Entrega y despliegue continuo (CD)

6. Entrega (Delivery) vs. Despliegue (Deployment)

La diferencia está en el **último paso**:



- **Entrega continua:** el pipeline deja todo listo y probado para producción, pero el pase final lo hace una **persona**, que aprieta un botón o aprueba el despliegue.
- **Despliegue continuo:** no hay intervención humana. Si el cambio pasa todas las validaciones, se publica en producción automáticamente.

Caso donde conviene la entrega continua: el sistema médico del TP anterior, o el sistema de un banco. Por regulaciones y auditorías, alguien responsable tiene que aprobar cada versión antes de que llegue a los usuarios, y a veces hay que coordinar el lanzamiento con capacitaciones o con una fecha acordada con el cliente. También sirve, por ejemplo, para no desplegar un sistema de ventas en pleno Black Friday.

7. Gestión de entornos

El código avanza por tres entornos, y en cada paso hay más confianza:

```
Dev  ──────────►  Staging  ──────────►  Prod
(desarrollo)     (copia de prod)       (usuarios reales)
```

- **Dev:** donde se integran los cambios del equipo. Puede ser inestable y tener datos de prueba.
- **Staging:** una copia lo más parecida posible a producción: misma configuración, mismos servicios y datos realistas (anonimizados).
- **Prod:** el entorno que usan los clientes.

La función de **Staging** es ser el ensayo general antes del paso final. Ahí se corren las pruebas de integración y de punta a punta, se hacen pruebas de carga, se valida la migración de la base de datos y el cliente o QA prueban la versión. Muchos errores aparecen recién en un entorno real (configuración distinta, otra versión de la base de datos), y Staging permite encontrarlos antes de que les pasen a los usuarios.

8. Principio de inmutabilidad

Un **artefacto inmutable** es el resultado de compilar o empaquetar el código una sola vez (por ejemplo, una imagen de Docker o un .jar), con una versión fija, y que **no se modifica nunca más**. Ese mismo archivo es el que pasa de Dev a Staging y de Staging a Prod. Lo único que cambia entre entornos es la configuración (variables de entorno), no el artefacto.

No hay que volver a compilar en cada entorno porque:

- **Lo probado es lo que se publica:** si se compila de nuevo para producción, podría bajar otra versión de una dependencia o usar otro compilador, y lo que llega a Prod ya no sería lo que se probó en Staging.
- **Seguridad:** se puede verificar con un hash que el artefacto no fue alterado en el camino. Si se recompila, alguien podría meter código malicioso en ese paso.
- **Rollback fácil:** las versiones anteriores siguen guardadas tal cual, así que volver atrás es solo desplegar el artefacto viejo.

Parte 4: Resiliencia y mitigación de riesgos

9. El mecanismo de rollback

A nivel de infraestructura, un rollback consiste en volver a poner en producción la versión anterior que funcionaba, en lugar de intentar corregir el error en el momento. No se toca el código: se le dice a la infraestructura que vuelva a usar el artefacto anterior (por ejemplo, la imagen app:2.4.1 en lugar de

app:2.5.0) o se redirige el tráfico a los servidores que todavía la tienen. Después, con el sistema estable, se investiga el error con calma.

Cómo ayudan los tags de SemVer: cada versión publicada tiene un tag y un artefacto guardado con ese mismo número. Entonces se sabe exactamente cuál es la última versión estable y su artefacto ya existe, no hay que compilar nada. El rollback es un solo comando, por ejemplo kubectl rollout undo o desplegar app:2.4.1, y tarda segundos o minutos.

10. Estrategias progresivas

	Blue-Green	Canary
Cómo funciona	Hay dos entornos de producción iguales. Blue tiene la versión actual y Green la nueva. Se prueba Green y después se cambia todo el tráfico de golpe.	La versión nueva se publica para un porcentaje chico de usuarios (por ejemplo 5%) y, si todo va bien, se va aumentando de a poco hasta el 100%.
Manejo del tráfico	Todo o nada: el 100% de los usuarios pasa de una versión a la otra en el mismo momento.	Gradual: el tráfico se reparte entre las dos versiones durante un tiempo.
Rollback	Inmediato: se vuelve a mandar el tráfico a Blue.	Se baja a 0% el tráfico de la versión nueva. Solo se vieron afectados los usuarios del grupo de prueba.
Costo	Necesita el doble de infraestructura.	Necesita buen monitoreo para comparar las dos versiones.

La que permite probar con un **grupo reducido de usuarios reales** antes del lanzamiento total es **Canary**. El nombre viene de los canarios que usaban los mineros para detectar gases: si algo sale mal, lo nota un grupo chico antes de que afecte a todos. En Blue-Green, el entorno nuevo se puede probar antes del cambio, pero los usuarios reales lo reciben todos juntos.